同济大学计算机系

操作系统课程实验报告

学　　号 2353761

姓　　名 王皙晶

专　　业 计算机科学与技术

授课老师 方钰

实验四：在 UNIX V6++中添加新的系统调用

一、实验目的

（1）在 UNIX V6++源代码中添加一个新的系统调用;

（2）进一步理解和掌握系统调用响应与处理的流程，特别是用户态到核心态的切换和栈帧的变化;

（3）进一步掌握 UNIX V6++重新编译及运行调试的方法;

二、实验结果

（1）完成实验 4.1，截图说明操作过程，掌握在 UNIX V6++中添加一个新的系统调用的方法，并总结出主要步骤。

添加一个新的系统调用的主要步骤有：

1. 在系统调用子程序入口表 m_SystemEntranceTable 赋值的程序代码中选择其中任何一个赋值为{ 0, &Sys_Nosys }的项（表示对应的系统调用目前未定义，为空项）来添加新的系统调用。用{ 系统调用所需参数个数, &系统调用处理子程序入口地址}来替换原来的{ 0, &Sys_Nosys }：

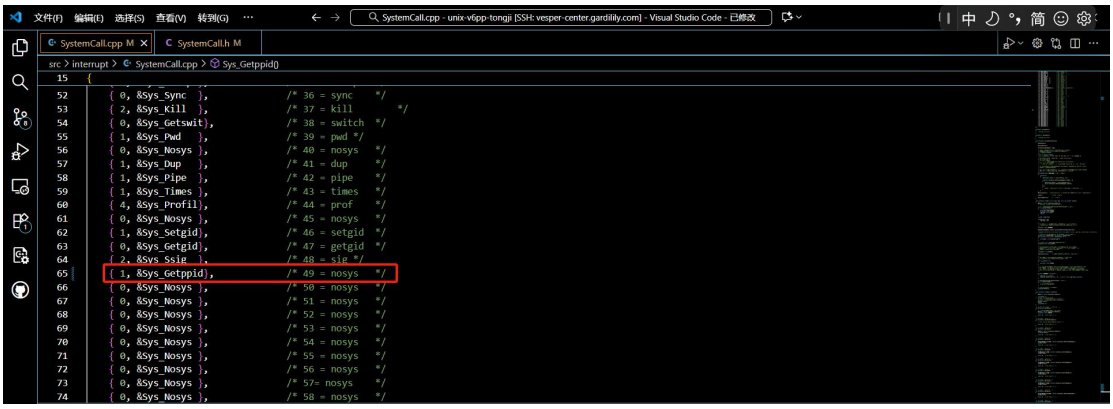

图 1

2. 在 SystemCall.h 文件中添加该系统调用处理子程序 Sys_Getppid 的声明。

图 2

3. 在 SystemCall.cpp 中添加 Sys_Getppid 的定义。

图 3

（2）完成实验 4.2~4.3，掌握在 UNIX V6++中添加库函数的方法，并编写测试程序，通过运行说明添加的系统调用的正确，截图说明主要操作步骤。

添加库函数的主要操作步骤为：

1. 找到 sys.h 文件，在其中加入名为 getppid 的库函数的声明；

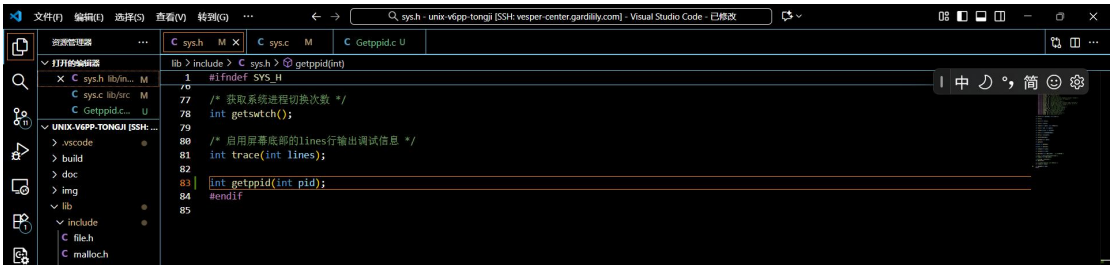

图 4

2. 在 sys.c 文件中添加库函数 getppid 的定义：

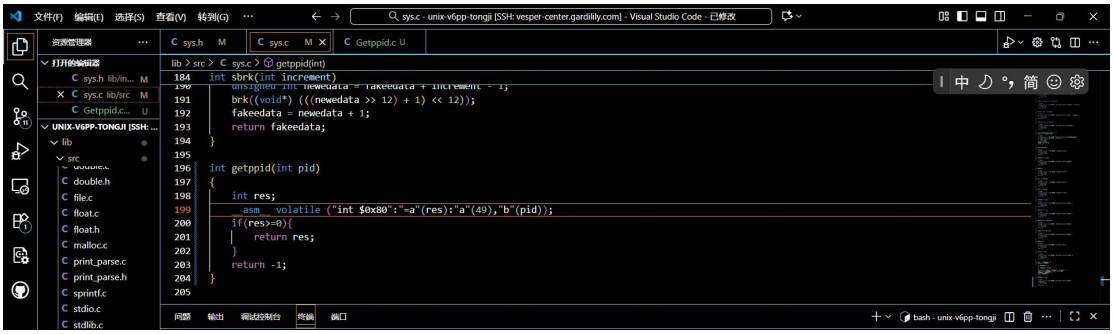

图 5

3. 重新编译 UNIX V6++。如果编译成功，则一个新的系统调用及和它对应的库函数已添加完毕：

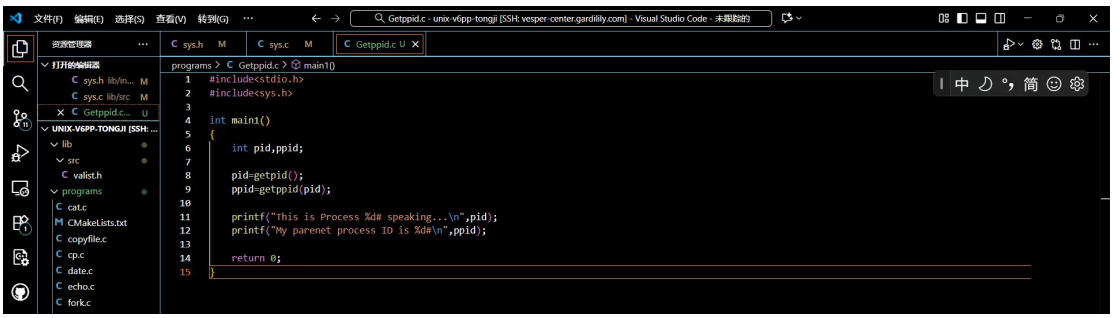

图 6

4. 编写并运行一个简单的测试程序来测试添加的新的系统调用能否正常工作。

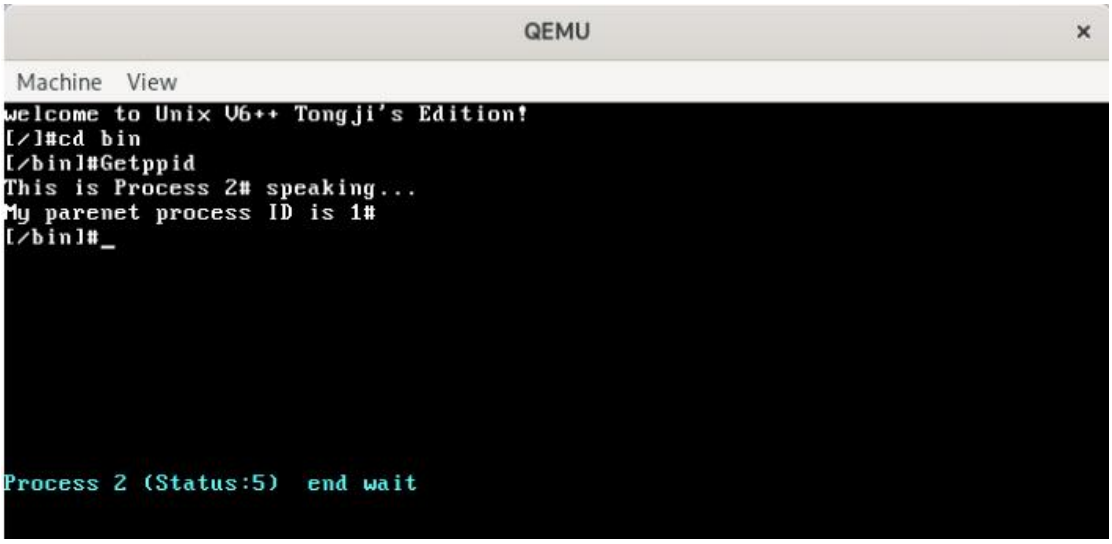

图 7

（3）完成实验 4.4，编写测试程序，通过设置合适的断点和观察内存，补充完成图 11，截图说明主要的调试过程和关键结果。

实验报告中图 11 主要是验证系统调用前后核心栈的变化，所以在系统调用前后设置断点，观察前后通用寄存器和内存地址的值是否一致。下图 8 展示了系统调用前后通用寄存器的值：

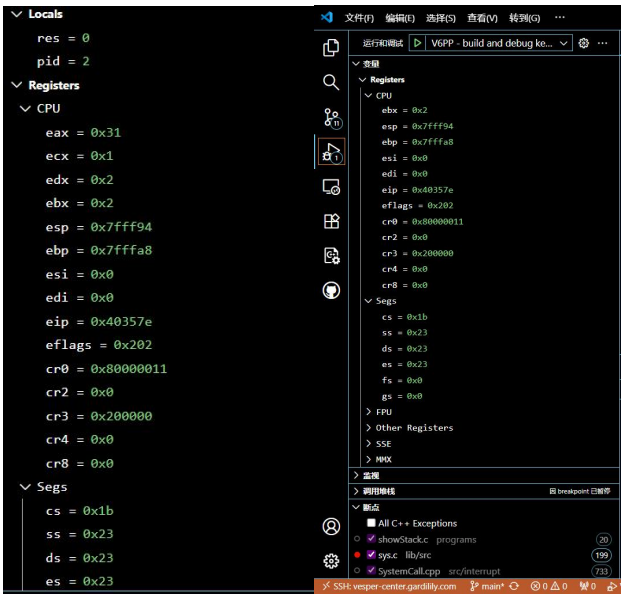

图 8

可以看出：ss=0x23 esp=0x7fff94 eflags=0x202 cs=0x1b eip=0x40357e eax=0x31 ebp=0x7fffa8 edi=0x0 esi=0x0 edx=0x2 ecx=0x1 ebx=0x1 es=0x23 ds=0x23 fs= 0x0 gs=0x0

图 9 所示为系统调用后查看的内存地址值，根据图 9 可以补充完成实验报告中的图，补充后的图如图 10 所示。

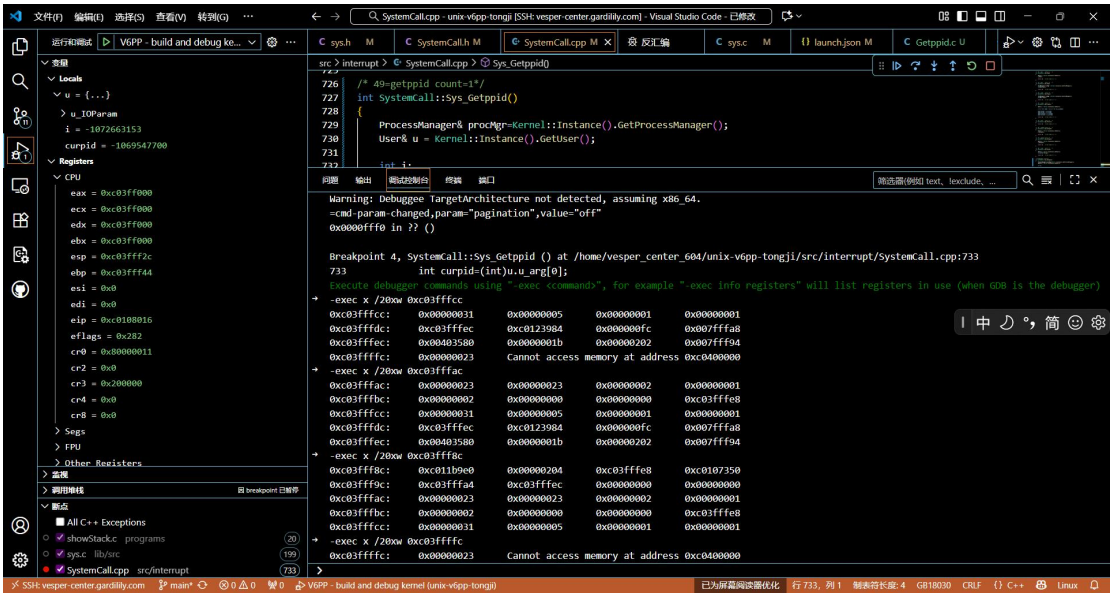

图 9

地址	核心栈内容	说明	
	0x00000204		系统调用处理程序的栈帧
	0xc03fffe8		
	0xc0107350		
	0xc03fffa4		软件现场（这里的值可以通过和图 8 的比对确认是否正确）
	0xc03fffec		
	0x00000000		
	0x00000000		
	0x00000023		
	0x00000023		
	0x00000002		
	0x00000001		
	0x00000002		
	0x00000000		
	0x00000000		
	0xc03fffe8		
0xC03FFFCC	0x00000031	保存 EAX 单元，值为系统调用号	
	0x00000005		系统调用入口程序栈帧
	0x00000001		
	0x00000001		
	0xc03fffec		
	0xc0123984		
	0x000000fc		
	0x007fffa8		
	0x00403580		硬件现场（这里的值可以通过和图 8 的比对确认是否正确）
	0x0000001b		
	0x00000202		
	0x007fff94		
0xC03FFFFC	0x00000023		
		已到达核心栈栈底，超出进程虚地址空间，将被拒绝访问	

图 10

值得一提的是，系统调用入口程序栈帧的 ebp 是调用前 ebp 的寄存器值，但是软件现场的 ebp 却是调用后的值，这两个 ebp 的值不同。经过逐步调试，可以看出 ebp 值不同的变化原因，如图 11 所示的汇编语句。

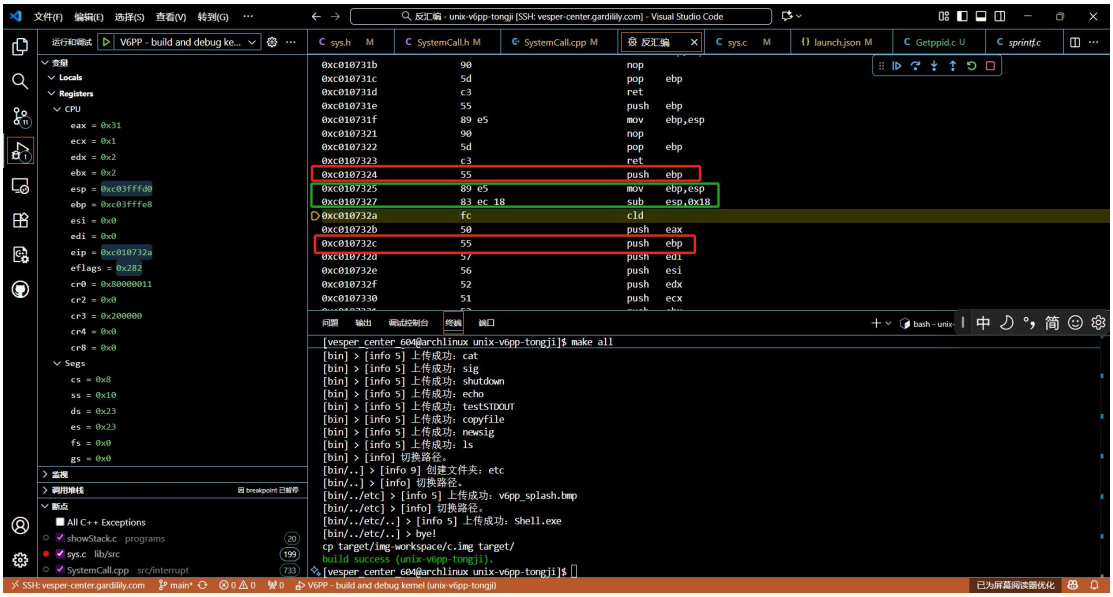

图 11